Module 1 — Thinking in Patterns (Foundations)

This module sets up the *method* every later module uses: what a pattern is, five fixed questions to ask of any design, and four quick calculations that show whether a design is big enough (or bigger than it needs to be). One example runs through the course: an online store that starts on a single database and grows to 100 million users, with each module tackling its next problem.

Course Introduction

Original lesson: Course Introduction on DesignGurus ↗

What it is

Most large systems are assembled from the same small set of recurring solutions, called **system design patterns**; the course covers more than 60. Once you know them, an unfamiliar architecture diagram stops being a pile of boxes: you can name each part ("a cache, a queue") and build new systems from pieces you understand instead of memorizing whole diagrams.

> **Analogy:** strong chess players don't memorize games; they learn recurring tactics (forks, pins) and spot them in new positions. Patterns are the forks and pins of system design.

Key terms

- **System design pattern:** a reusable solution to a recurring problem, bundled with the price you pay for using it.
- **Pattern vs technology:** a technology is a product; a pattern is the idea it implements. *Redis* (an in-memory data store) is a technology. *Cache-aside* is a pattern: keep a fast temporary copy of frequently used data (a **cache**), look there first, and go to the slower database only when the copy lacks the data (a **cache miss**). Products come and go; patterns keep their value.
- **Server / database / HTTP request / client:** all you need going in. A server answers requests over the network; a database stores data that survives restarts; an HTTP request is the standard message a **client** (browser, app, or another server) sends to a web server. If any feel shaky, see System Design Fundamentals.

Who it's for

Early- to mid-career software engineers, plus data and AI engineers whose work spans many machines and hits the same problems. No **distributed systems** background is assumed (systems of many computers cooperating over a network that must keep working when some fail); every term is defined at first use.

The twelve modules

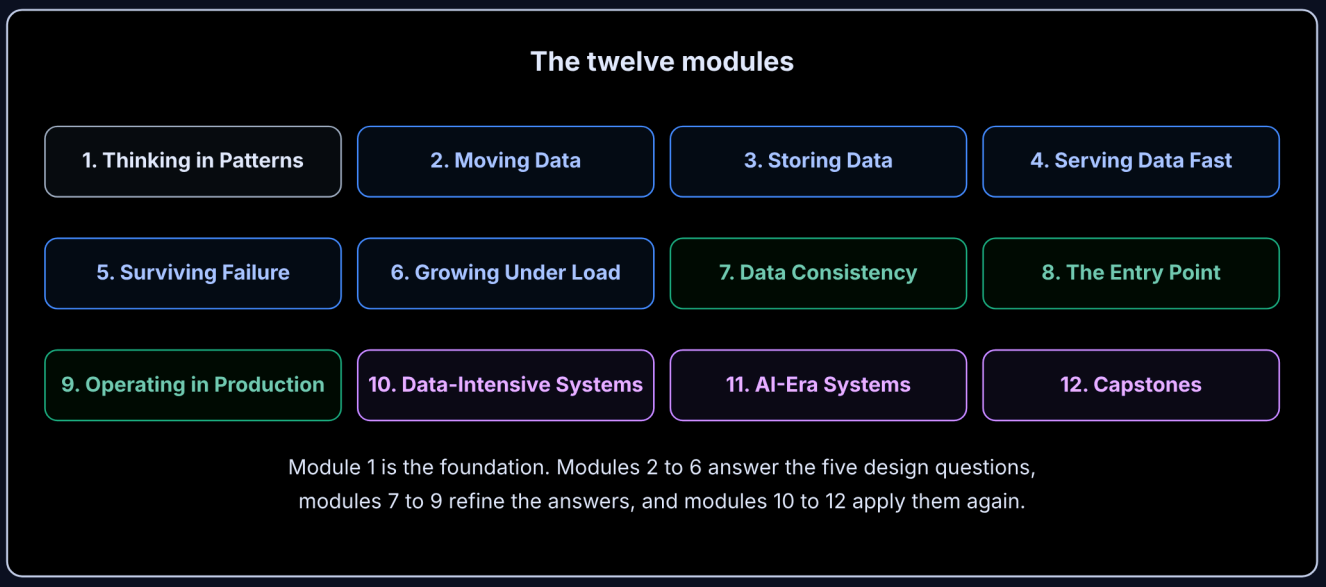


Figure — The twelve modules (course diagram)

Module 1 lays the groundwork, Modules 2–6 each answer one of the five design questions below, 7–9 sharpen those answers, and 10–12 apply everything to larger systems.

Module	What it covers
1. Thinking in Patterns	What a pattern is, plus the simple math used everywhere
2. Moving Data	How services talk: request-response, queues, pub/sub, streams
3. Storing Data	Replication (copies on several machines), sharding (splitting data across machines) and logs
4. Serving Data Fast	Caching and its price
5. Surviving Failure	Calls that hang, fail, or arrive twice
6. Growing Under Load	Scaling, load balancing, connection pooling
7. Keeping Data Consistent	Transactions (all-or-nothing changes), sagas (multi-step workflows with undo steps), quorums (majority votes)
8. The Entry Point	Gateways (front door for requests), rate limits (caps on requests/s), CDNs (servers near users holding copies of content), proxies (middlemen forwarding requests)
9. Operating in Production	Deploying, monitoring, rolling back
10. Data-Intensive Systems	Processing events in batches (large chunks) and streams (one by one as they arrive)
11. AI-Era Systems	The same patterns inside LLM (large language model, like ChatGPT) and ML (machine learning) systems
12. Capstones	Four full designs pulling everything together

Added in these notes (not part of the course): five extra lessons in modules 6, 8, 9 and 10, and six case-study modules — 13 Building Blocks, 14 Messaging and Media, 15 Location and Real-Time, 16 Infrastructure, 17 Money and Reservations and 18 Client and Mobile — applying the same patterns to classic "design X" prompts.

How each lesson works

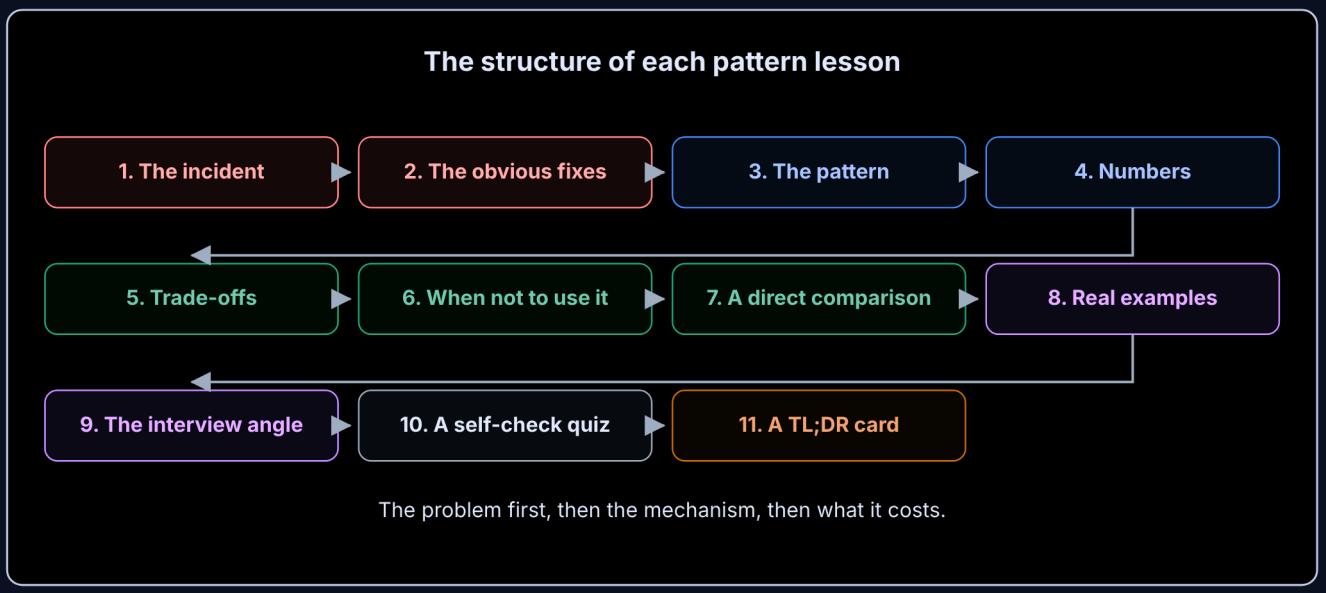


Figure — The structure of each pattern lesson (course diagram)

Every pattern lesson follows the same 11 steps: problem first, then the fix, then its cost.

1. **The incident:** a believable failure in **production** (the live system real users use), with real numbers.
2. **The obvious fixes** engineers try first, and why they fall short.
3. **The pattern**, step by step.
4. **Walkthrough with numbers** on the same incident.
5. **Trade-offs:** the cost, delay or complexity it adds.
6. **When not to use it.**
7. **Direct comparison** with the pattern it's most often confused with.
8. **Real-world examples** from production systems.
9. **Interview angle**, with a rehearsed 30-second answer.
10. **Self-check quiz.**
11. **TL;DR card** ("too long; didn't read"): together the cards form a night-before-the-interview review deck.

Lessons are full or compact (simpler patterns have fewer decisions). Each module ends with a **mini-capstone**: a small design using that module's patterns together.

This module

Three lessons build the method: **Thinking in Patterns** (what a pattern is and why it beats memorized architectures), **The Five Questions** (turning a fuzzy prompt into a plan), and **Back-of-the-Envelope Math** (four reusable calculations). Request-Response, next module, is the first place the method is applied.

Thinking in Patterns

Original lesson: [Thinking in Patterns on DesignGurus ↗]

What it is

A way of reasoning about design: instead of recalling whole architectures, split the system into *problems you already know*, pick the standard solution for each, and state what each costs. **Analogy:** a doctor doesn't memorize patients; they map symptoms to known conditions and know each treatment's side effects.

The problem: two candidates, one question

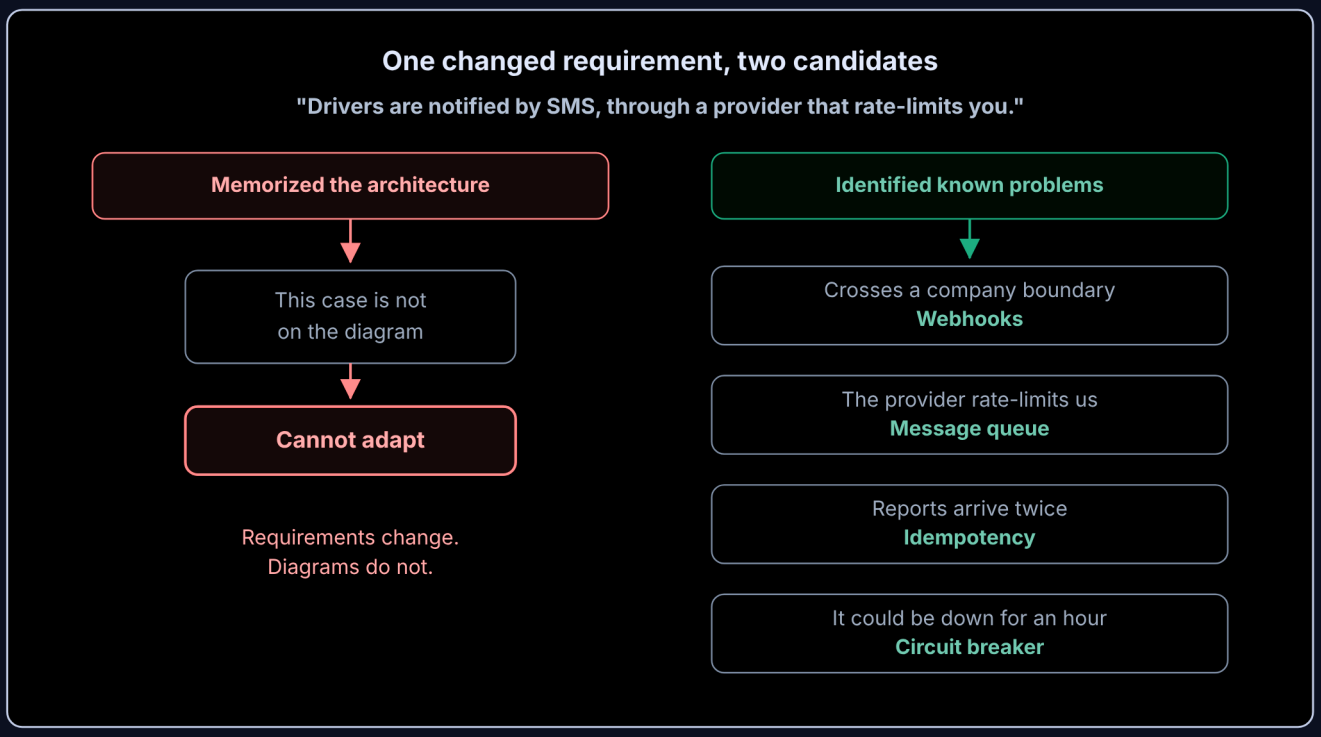


Figure — One changed requirement, two candidates (course diagram)

Two candidates get "design a ride-sharing service." The first redraws Uber's published architecture from memory. Then the interviewer changes one requirement: *driver alerts now go out as SMS through a third-party provider that rate-limits you.* The memorized diagram has no answer, so the first candidate is stuck.

The second splits the change into four known problems:

- **Work crosses into another company.** The provider sends delivery reports back as **webhooks**: HTTP requests the provider makes into *your* server.
- **The provider rate-limits you** (caps requests per second), so messages go on a **message queue** and are sent at a steady, controlled rate.
- **A report may arrive twice**, so handling must be **idempotent**: doing it twice leaves the same result as doing it once.
- **The provider may be down for an hour**, so a **circuit breaker** pauses calls to it and the app alerts users in-app instead.

Four patterns, each with its cost stated; when the next requirement changes, the same reasoning runs again. As the figure says: *requirements change; diagrams do not* — on the job as much as in interviews.

What a pattern is

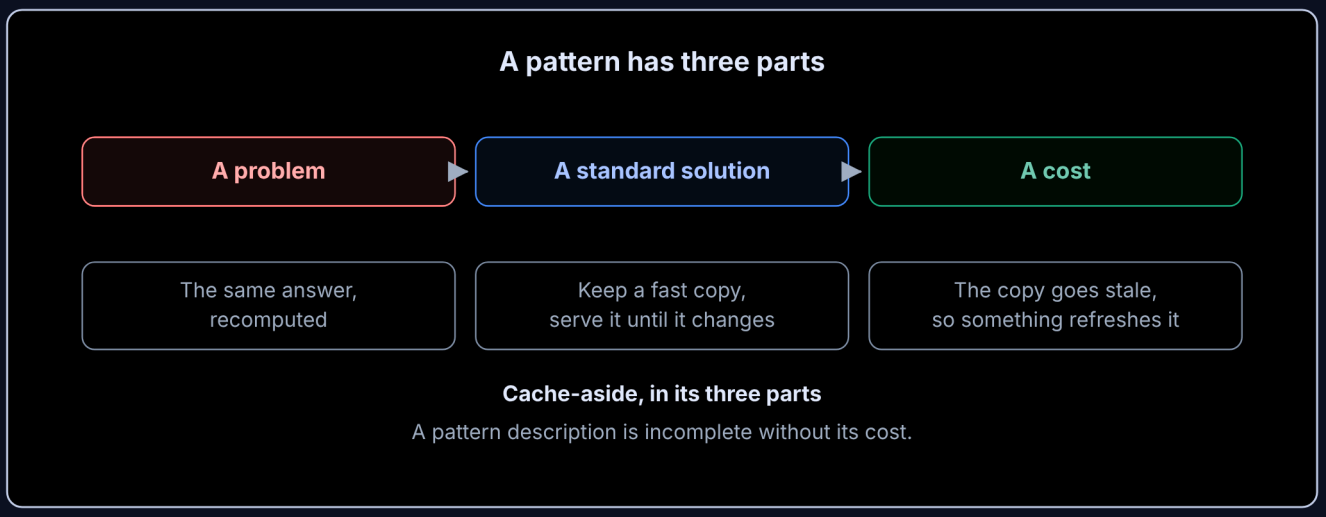


Figure — A pattern has three parts (course diagram)

Every pattern has three parts (shown with cache-aside); missing one means the description is incomplete:

1. **A problem in general terms.** Not "Instagram is slow" but "the same unchanged answer is recomputed thousands of times," which any product can have.
2. **A standard solution.** Keep a fast copy of the answer and serve it until the answer changes.
3. **A cost.** The copy can go **stale** (out of date), so something must refresh it. The cost is part of the pattern, not a detail for later.

Redis, Kafka (a message-log product) and Postgres (a relational database) don't define patterns; they are products that implement them well today.

Patterns predate software. Paper bank ledgers are event sourcing (record every change, derive the balance). Ship bulkheads keep one flooded compartment from sinking the ship. Post offices had a dead letter process for undeliverable mail. About sixty patterns cover so much because every system hits the same limits: networks break, machines have finite capacity, copies drift apart, load arrives unevenly.

Patterns combine into architectures

An architecture is several patterns working together; Netflix is roughly fifteen. Caches cut repeated reads, queues decouple one **service** (a separately running program owning one job, such as payments) from another, circuit breakers guard calls to its **dependencies** (systems it needs to do its job), and replicas keep extra data copies. So "design X" becomes three steps:

1. **Find the problems.** A news feed has **fan-out** (one post must reach many followers) and **hot keys** (one item draws a huge share of traffic); checkout has payment and third-party problems.
2. **Pick a pattern per problem, then check interactions. Retries** (automatically resending a failed request) need idempotency or a retried payment double-charges. Caches need an expiry rule. **Shards** (pieces of a database on different machines) need a **shard key**: the field that decides which piece stores each row.
3. **Name every cost**, and decline a pattern when the problem isn't big enough to justify it.

Each module's capstone runs these steps on a small system; the last module runs them on four full designs.

Every pattern is a trade

The one habit to keep: no pattern makes everything better.

Pattern	What it gives	What it costs
Queue	Absorbs traffic bursts	Adds waiting time; a message may arrive twice
Cache	Fast reads	May serve out-of-date data
Sharding	Room for more writes	Cross-shard queries become hard
Circuit breaker	Fails fast instead of hanging	May "trip" when the dependency could still have replied

Hence every lesson's "When Not to Use It." Weak answer: "We'll use Kafka here." Strong answer: "A queue fits because no one waits on this work. Queues can deliver twice, so the **consumer** (the program reading from the queue) dedupes; they can reorder messages, which this operation tolerates." That covers **problem, pattern and cost**. Juniors know what patterns do; seniors also know each one's price and when to turn it down.

How interviews test patterns

Interviewers rarely ask for definitions. They test:

- **Spotting problems from symptoms.** Every **health check** (an automatic "are you alive?" probe) is green, yet one feature is starved of resources: think "bulkhead" (give each feature its own resource pool) before they say it. That's why lessons open with an incident.
- **Pattern interactions.** "Could a retry charge someone twice?" "What happens when your busiest cache key expires?"
- **Defending choices.** "Why not X everywhere?" means "what does X cost?" Answers without trade-offs sound memorized.

How to study

- **Recognize, don't recite:** at each incident, diagnose before reading the fix.
- **Pair every pattern with its cost** in one sentence (the TL;DR cards help).
- **Practice on apps you use daily** with the five questions (next lesson).
- **Say the cost out loud** every time you name a pattern.

Interview one-liner

"I don't recall architectures. I split the system into problems I already know, choose one pattern per problem, look at how those patterns affect each other, and name the cost of each. A changed requirement just means rerunning the same reasoning."

TL;DR

- **The claim:** ~60 recurring patterns underlie every system design; knowing them beats memorizing architectures, because requirements move and diagrams can't.
- **A pattern is:** problem + standard solution + cost. Technologies implement patterns; they don't replace them.
- **Designing means:** find the problems, map each to a pattern, check interactions, state every cost.
- **The senior habit:** every pattern is a trade-off; interviews reward knowing the price and when to decline.
- **Study by:** diagnosing incidents before reading fixes, and never naming a pattern without its cost.

The Five Questions

Original lesson: [The Five Questions on DesignGurus ↗]

What it is

A checklist of five questions, always in the same order, that you point at any system. Each question surfaces problems; each problem leads to patterns. **Analogy:** a pilot's pre-takeoff checklist — the pilot can fly; the checklist ensures nothing is forgotten under stress.

The problem

"Design a URL shortener" (a service like bit.ly that maps a long URL to a short code and redirects visitors), then silence. Most candidates freeze not from lack of knowledge but lack of procedure. A fixed order of questions means you always know the next step; this lesson shows *where to look* for patterns.

Step 0 — get the numbers first: user count, **RPS** (requests per second), data size, **read:write ratio** (reads per write), and the cost of an outage. A design for 100 users won't suit 100 million (the next lesson shows how to estimate these quickly).

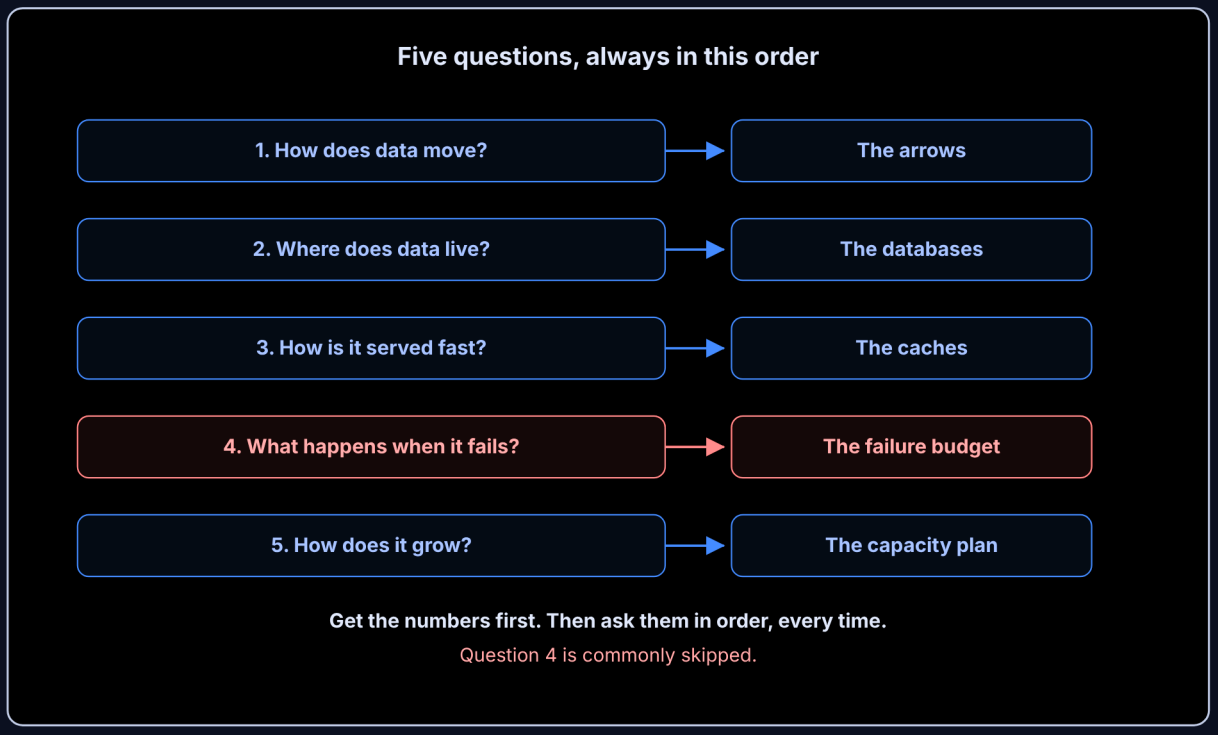


Figure — Five questions, in order (course diagram)

Each question fills in one part of the diagram: arrows, databases, caches, failure budget, capacity plan. Note the figure's warning: question 4 is the most skipped.

The five questions, one by one

1. How does data move? → the arrows. Who talks to whom, and does the caller wait? Must the work happen exactly once? Do several services care about the same fact? Does the call leave your company? Must data flow continuously? Options (Module 2):

- **Request-response** (default): the caller waits for the reply.
- **Message queue**: non-urgent work is held until a worker takes it.
- **Pub/sub** (publish-subscribe): one event delivered to every interested service.
- **Webhooks**: one company's system makes HTTP calls into another's.
- **Streams**: the server pushes data continuously.

Example: "Order confirmation must be immediate, so a direct call. Fulfillment can lag, so a queue. Five teams care about each order event, so we publish it."

2. Where does data live? → the databases. Which store is the **source of truth** (the official copy), how big it grows and how fast, the read:write ratio, and whether you need full history or only current state. Options (Module 3):

- **Replicas**: extra database copies, added when reads outgrow one machine.

- **Sharding**: splitting rows across machines, when writes or total size outgrow one.
- **Consistent hashing**: changing the shard count without moving most data.
- Log patterns when history matters: a **write-ahead log (WAL)** records each change before applying it; **event sourcing** stores every event instead of only the final state; **CQRS** uses separate models for writes and reads.

Example: "Customers dispute charges, so we keep order events, not just current state."

3. How is it served fast? → the caches. Which reads repeat, how stale each data type may be, the **hit ratio** (share of reads the cache answers), and what breaks when a hot entry expires. Start with **cache-aside**, use write-based variants for frequently changing data, and add **stampede prevention** for when many requests miss the same hot entry at once. The caching module's key decision: give each data type a **staleness budget** (how old it may be) and pick a mechanism that honors it.

Example: "Descriptions can be an hour old. Prices a few seconds, if updates **invalidate** the cache (delete or mark the stale copy so the next read fetches fresh). At checkout the price must be exact, so it reads the database."

4. What happens when it fails? → the failure budget. For *every arrow* from question 1: what if the other side never answers, fails once, or is down ten minutes? Could a retry cause a double effect such as a charge? What does the user see? Toolbox (Module 5):

- **Timeout**: stop waiting after a deadline.
- **Retry with backoff**: retry, waiting longer each time.
- **Idempotency**: a repeated request can't repeat the effect.
- **Circuit breaker**: stop calling a service during a long outage.
- **Bulkhead**: keep one broken feature from consuming shared resources.
- **Graceful degradation**: return a smaller result instead of an error.

Example: "If the payment provider slows, we cut the call at twice its normal worst-case **latency** (time one call takes end to end). Retries carry **idempotency keys** (a unique request ID the receiver uses to ignore repeats), so no double charge. If failures persist, the breaker opens, and we save the cart and email the customer."

5. How does it grow? → the capacity plan. At 10x traffic, what breaks first: reads, writes, storage, or one hot key? What scales by adding machines, and what needs a redesign? Module 6 covers **load balancing** (spreading requests across servers), **autoscaling** (adding/removing servers automatically with traffic) and **horizontal scaling** (growing by adding machines).

Example: "Replicas and caching handle reads, but the single **primary** (the one server accepting writes; replicas copy from it) caps writes. Next step is sharding by customer ID, planned before the primary hits its limit."

The questions map to Modules 2–6 in order. Later modules add consistency, the entry point and operations; the data and AI modules run the same questions on **pipelines** (chains of steps moving and transforming data) and ML systems.

Walkthrough: the URL shortener in about three minutes

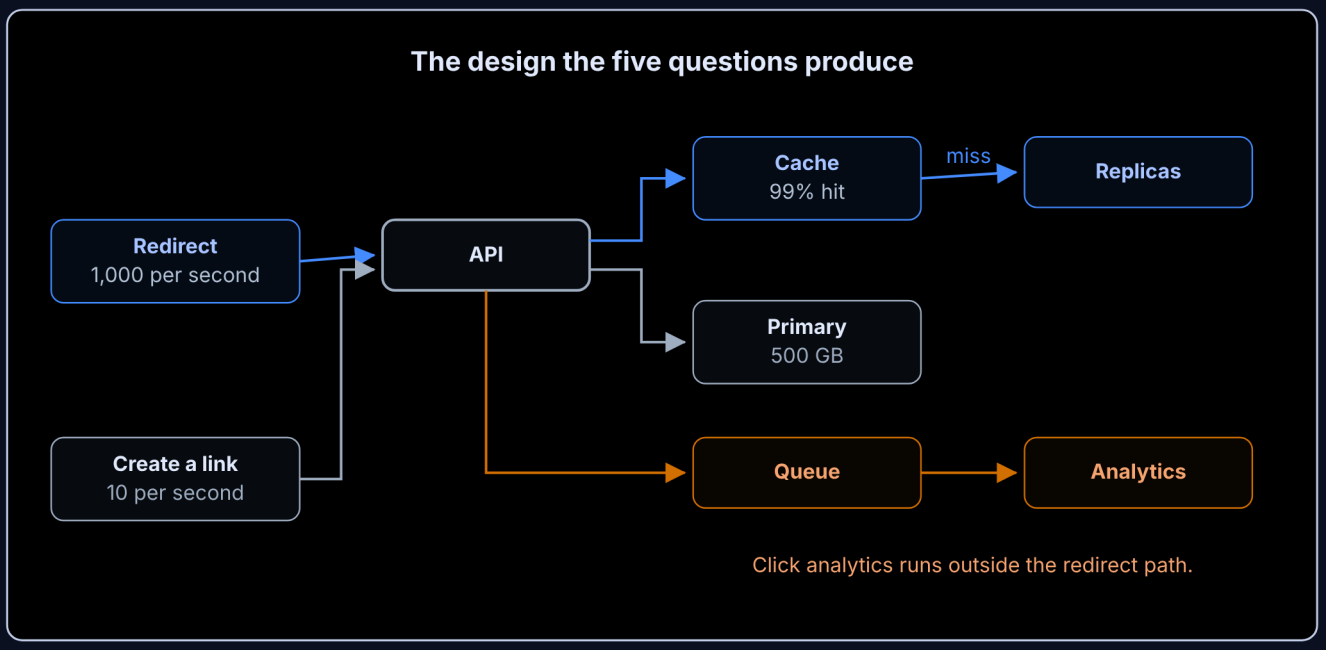


Figure — The design the five questions produce (course diagram)

End state (figure): redirects at 1,000/s hit a cache with a 99% hit ratio; misses go to replicas behind a 500 GB primary; link creation (~10/s) goes to the primary; click analytics flows through a queue *off* the redirect path.

- **Move:** one write path (long URL in, short code out) and one read-heavy redirect path, ~100 reads per write, both direct calls. Click analytics is work nobody waits on, so it's queued and never slows a redirect.
- **Live:** tiny rows (two URLs and a counter), potentially billions. Primary plus replicas for now; at billions of rows, shard by short code, since every lookup includes it. No history needed.
- **Fast:** a code always maps to the same URL, so cache-aside with a long **TTL** (time-to-live: how long an entry stays cached). A few links get most clicks, so hot links get stampede prevention and shouldn't expire in normal operation.
- **Fail:** redirects need very high **availability** (share of time a service answers correctly), so a cache miss reads a replica. Link creation may answer "try again shortly" because its outage hurts less. Each path gets its own reliability target.
- **Grow:** replicas and cache nodes absorb reads; writes are few. The real risk is **key generation**: random codes need **collision** checks (two links getting the same code); counter-based codes need coordination so two servers never issue the same one. Flag it as future work with both options.

Three minutes yields separate read/write paths, storage with a chosen shard key, a caching plan, per-path reliability targets, and one flagged growth risk. When a requirement changes, rerun the questions.

Usage tips

- **Announce the questions as your agenda:** "First how data moves, then storage, caching, failures, growth."
- **Use them as a checklist** when stuck or when time remains.
- **Ask question 4 for every arrow**: it's the most skipped and draws the most follow-ups.
- **Bring them to design reviews at work.** "What if the other service never answers?" is among the most useful questions in distributed systems.

Interview one-liner

"After getting the numbers, I go through five questions in order: how data moves, where it lives, how it's served fast, what happens when it fails, and how it grows. Each surfaces problems that map to patterns, and the fixed order means nothing gets skipped."

TL;DR

- **Procedure:** numbers first, then five fixed questions (move, live, fast, fail, grow).
- **Why it works:** questions surface problems, problems lead to patterns, fixed order guarantees coverage.
- **Structure:** questions map to Modules 2–6; later modules refine the answers.
- **Most-skipped:** Q4, failure handling — where designs become dependable and interviewers push hardest.
- **Use it as:** your spoken agenda, your checklist when stuck, your go-to review question.

Back-of-Envelope Math

Original lesson: Back-of-Envelope Math on DesignGurus ↗

What it is

Fast, deliberately rough calculations that give the *order of magnitude* a system needs: one machine or ten? 600 requests in flight or 60,000? Does the data fit in memory? **Analogy:** "about 5 hours and two tanks of gas" is enough to choose between driving and flying. The fundamentals are covered in Module 0; this lesson keeps the four formulas the course reuses.

Why engineers estimate

A rough answer in 30 seconds shapes today's design; a precise one next week doesn't. You can't answer the five questions without numbers, and interviewers ask "how much storage?" expecting stated assumptions, rounded numbers and visible steps.

Round hard. A day's 86,400 seconds → **100,000**; a month's ~2.5 million → **3 million**. You want **orders of magnitude** (powers of ten), not decimals. When direction matters, round demand up and capacity down to leave headroom.

Numbers to memorize

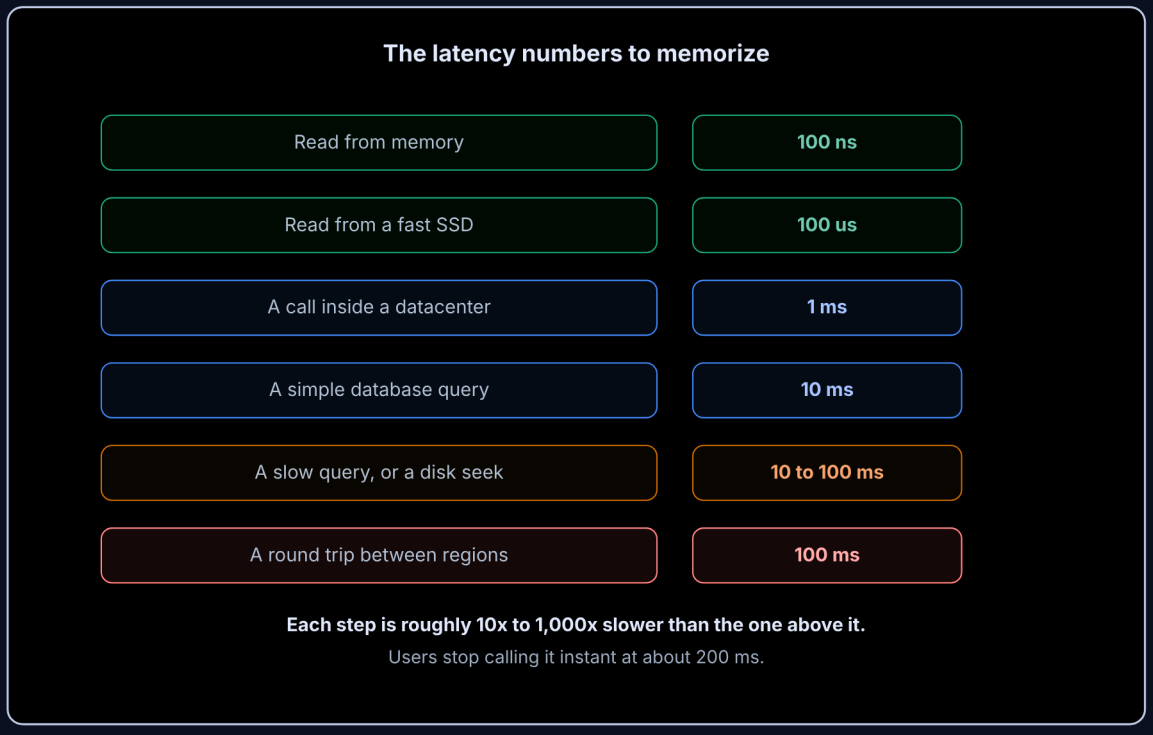


Figure — The latency numbers to memorize (course diagram)

Latency is how long an operation takes. Each row is roughly 10–1,000x slower than the one above (fuller table: Module 0):

Operation	Time
Read from RAM (fast working memory)	~100 ns
Read from fast SSD (solid-state drive, flash storage)	~100 µs
Network call within one datacenter (a building full of servers), e.g. a cache read	~0.5–1 ms
Simple database query	~5–15 ms
Spinning-disk seek, or a slow query	~10–100 ms
Round trip (request there, reply back) between regions (geographic areas with their own datacenters)	~100 ms
Longest delay that still feels "instant"	~100–200 ms

Three design facts follow:

- A cache hit (~1 ms) is ~10–15x faster than a DB query (~10–15 ms): why caching helps.
- Sequential disk writes are much faster than scattered ones: why the write-ahead log works.
- Each *sequential* cross-region round trip adds ~100 ms, so a few consume the whole response budget.

Sizes: a character ≈ 1 byte; an ID ≈ 16–36 bytes; a DB row ≈ 1 KB; an image ≈ 100 KB–1 MB. Each unit is 1,000x the last (KB → MB → GB → TB), so 1 million 1 KB rows ≈ 1 GB and 1 billion ≈ 1 TB.

The four calculations

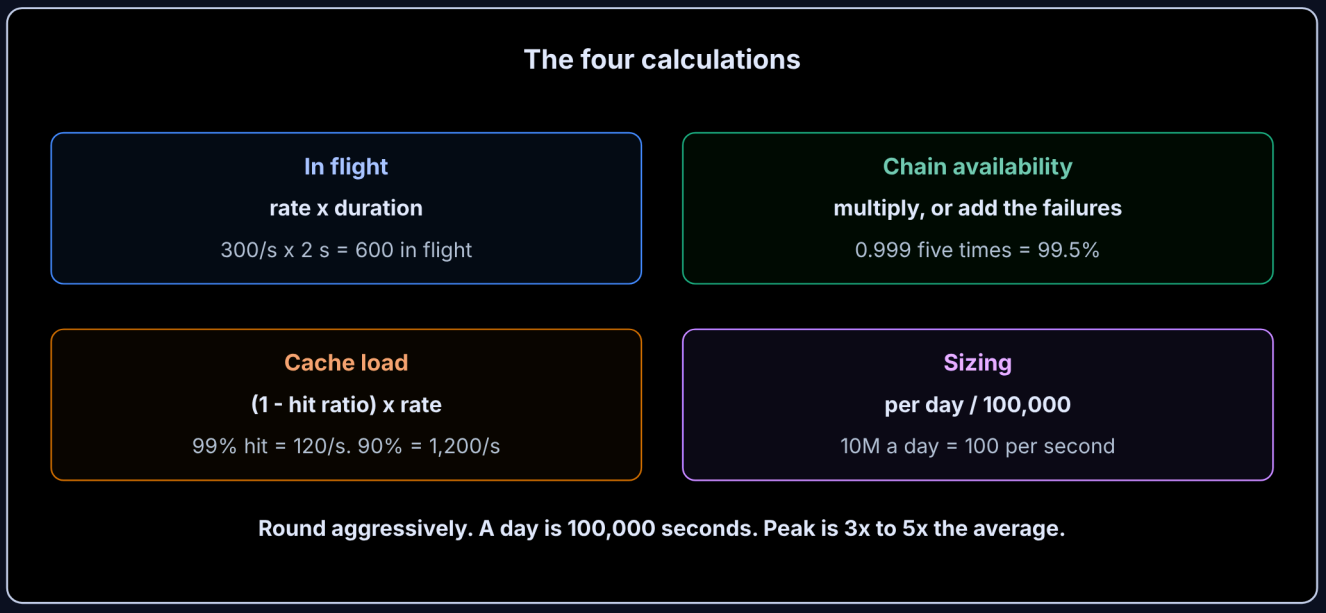


Figure — The four calculations (course diagram)

1. Rate × duration = work in progress (Little's law). `things in flight = arrival rate × duration`. 300 requests/s × 2 s each = **600 active requests**, each holding a **thread** (an independent line of execution; servers run many) or a database connection, so a **connection pool** (a fixed set of reusable open DB connections) of 200 is too small. The same formula gives a queue's **drain time** (time to clear a backlog): 552,000 messages at 400/s ≈ 1,400 s ≈ **23 minutes**. It's the course's most-used calculation, sizing thread pools, connection pools, bulkhead limits and backlog drain; name **Little's law** in interviews (Module 0).

2. Availability drops along a chain. Availability is the share of time a service answers correctly. When a request needs several parts in series, multiply their availabilities; the result is always below the weakest part. Five dependencies at 99.9%: 0.999^5 ≈ **99.5%**. Shortcut for small failure rates: *add* them (5 × 0.1% ≈ 0.5% failure).

Availability	Downtime per year	Per month
99%	~3.7 days	~7 hours
99.9%	~9 hours	~43 minutes
99.99%	~53 minutes	~4 minutes
99.999%	~5 minutes	~26 seconds

Each extra "nine" costs about **10x** the engineering effort of the last, and every required dependency can only *pull the total down*. So the practical fix is **fewer required dependencies**, not perfect ones; graceful degradation builds on this by returning a smaller but useful answer when an optional dependency fails. (More: Module 0.)

3. Cache hit ratio. With hit ratio *h* (share of requests the cache answers):

- `average latency = h × fast + (1 − h) × (fast + slow)` (a miss pays for the cache check *and* the DB)
- `load on the database = (1 − h) × request rate`

At 12,000 RPS, 1 ms hits and 15 ms DB reads:

- h = 99% → ~1.15 ms average; DB gets 1% × 12,000 = **120 QPS** (queries per second).
- h = 90% → ~2.5 ms average; DB gets **1,200 QPS**, ten times the load.

99% → 90% isn't "9% worse": the *miss* rate goes 1% → 10%, so the DB sees 10x traffic. Judge caches by their misses. It also explains a **cache stampede**: when a hot entry expires, its hit ratio briefly drops to ~0 and nearly every request hits the DB at once.

4. Sizing.

- `per second = per day / 100,000`: 10M orders/day ≈ **100/s average**.
- Plan for a **peak of 3–5x average**: 300–500/s.
- `storage = records × size × retention`: 10M/day × 1 KB × 365 ≈ **3.7 TB/year** — fits one ordinary database server, so this estimate *saves* you from sharding prematurely.

Walkthrough: sizing the URL shortener (~60 seconds, five numbers)

State each assumption aloud:

- "Assume **100M redirects/day** ÷ 100,000 = **1,000/s** average; plan for **~4,000/s** peak."
- "Each mapping is two URLs plus **metadata** (creation date, owner): **~1 KB**. **500M links** ≈ **500 GB**. One database handles that today, so sharding waits."
- "~100:1 reads to writes, and a few links get most traffic. Caching the top 1% is **5M rows ≈ 5 GB**; at a 99% hit ratio only **~10 queries/s** reach the DB."
- "At peak, 4,000/s × 2 ms = **8 requests in flight**. So: one strong database, one replica, a cache, careful failure handling."

Every component now has a capacity-based reason, and nothing is built for demand that hasn't arrived.

Traps

- **90% is not close to 99%:** it means 10x the misses.
- **Each extra nine costs ~10x**, and long dependency chains only lower availability, so keep them short.

Using estimation in interviews

- **State assumptions before calculating** ("1 KB per record, 10M records a day"); an interviewer can correct a spoken assumption, not a silent one.
- **Round, multiply, then check units:** rate × seconds = count; bytes × rows = storage. Wrong units, wrong math.
- **Let the numbers remove components.** "At this scale we don't need that yet" shows cost awareness, backed by arithmetic.

Interview one-liner

"I round hard (100k seconds per day, a gigabyte per million 1 KB rows, peak at 3–5x average) and use four formulas: rate × duration for concurrency, multiplied availabilities for chains, hit ratio for cache latency and DB load, and records × size × retention for storage. That tells me what the design needs and what it doesn't need yet."

TL;DR

- **Rule:** round hard; a day = 100,000 s; 1M × 1 KB = 1 GB; peak = 3–5x average.
- **Table:** RAM ~100 ns, SSD ~100 µs, same-datacenter hop ~1 ms, DB query ~10 ms, cross-region ~100 ms, "instant" ends at ~100–200 ms.
- **Four calculations:** in flight = rate × duration; chain availability = multiply (or add failure rates); cache latency and DB load from hit ratio; per second = per day / 100,000 and storage = rows × bytes × retention.
- **Traps:** 90% ≠ 99% (10x the misses); each nine ~10x the cost; chains only lower availability.
- **In interviews:** state assumptions, check units, let arithmetic rule out over-engineering.